PRODUCT DESIGN AND DEVELOPMENT

System Architecture

Overall System Architecture

The smart clinic management system is designed based on a robust multi-layer architecture to ensure high scalability, strict security, and ease of maintenance. The ecosystem comprises several key modules: a cross-platform frontend application, a central backend API, an independent AI inference engine, a Blockchain network, and a distributed database.

System Layer

To increase modularity and ensure the Separation of Concerns, the system is strictly divided into functional layers:

- **Presentation Layer:** Acts as the primary user interface, allowing patients, doctors, and administrators to interact seamlessly through a mobile application and a web administration dashboard. It handles data rendering, captures user inputs, and displays asynchronous feedback.
- **API Gateway:** Serves as the central access point and intermediary between the frontend and backend services. It is responsible for request routing, load balancing, Token-based Authentication, and enforcing rate limits to mitigate Denial of Service (DoS) attacks.
- **Application & Business Logic Layer:** This layer processes RESTful API requests, enforces Role-Based Access Control (RBAC), and manages core business logic (appointment scheduling, patient data analysis). It completely decouples business rules from the controllers, ensuring high maintainability.
- **Data Access & Data Layer:** Utilizing the Repository pattern and an Object-Data Mapper (ODM/ORM), the Data Access Layer optimizes queries and manages transactions. The underlying Data Layer involves multiple storage platforms: a NoSQL database for centralized patient records, an in-memory database for caching frequently accessed data, a Blockchain ledger for storing immutable hashes and audit logs, and a dedicated file storage environment for Machine Learning models.

The structural overview and the interaction flow between these functional layers are illustrated in **Figure 3.1** below

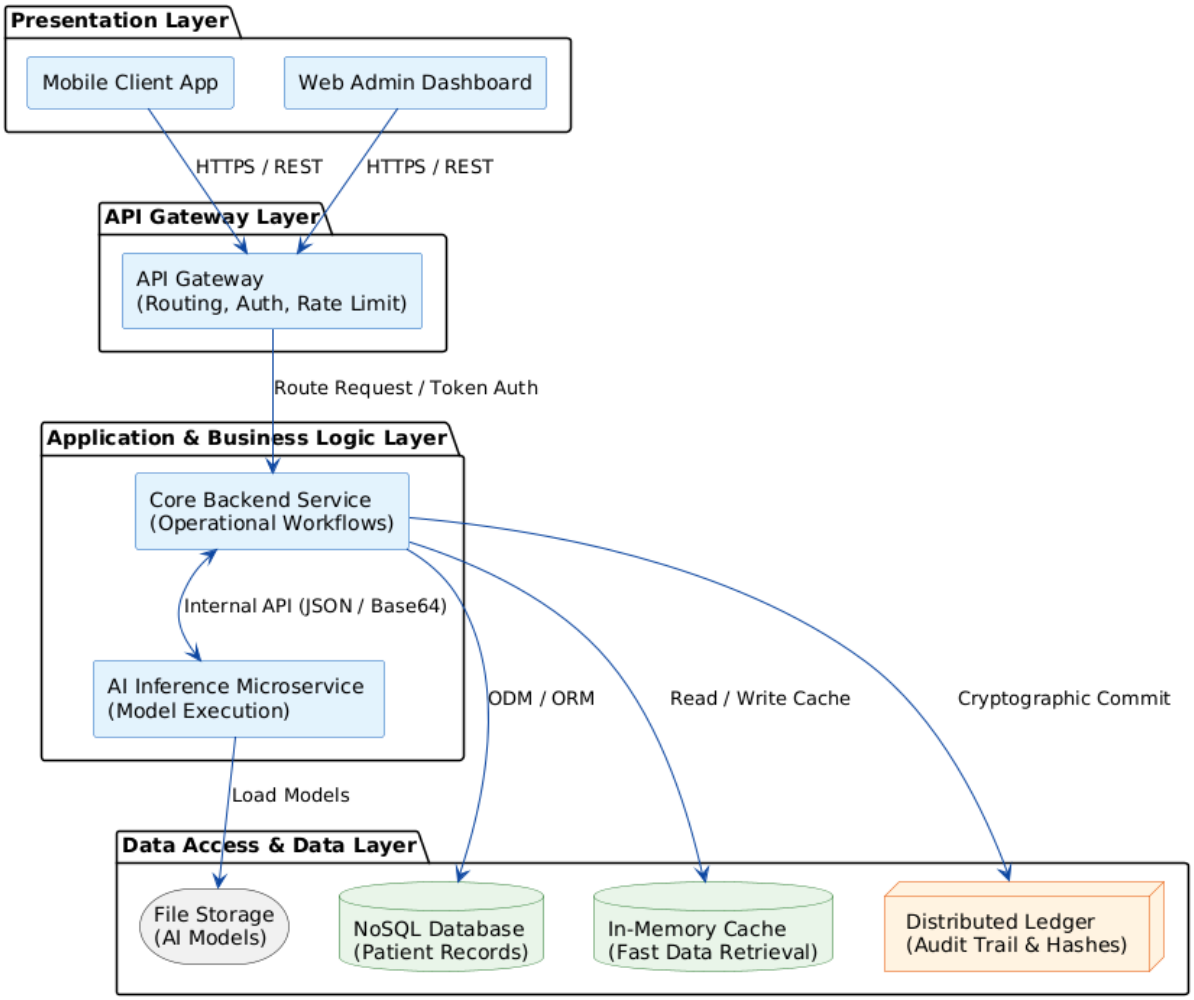

Figure 3.1: Multi-layer System Architecture Diagram

Microservices and API Architecture

To prevent system bottlenecks typically found in monolithic applications, the project adopts a Microservices architecture. Core operational workflows (patient management, appointment scheduling, billing) are handled by a Core Backend Service.

However, computationally intensive tasks are decoupled. The Artificial Intelligence module operates as an independent **API Microservice**. These isolated services communicate via standard RESTful APIs using structured JSON payloads. When a doctor uploads a medical image, the core backend acts as a client, forwarding a Base64-encoded payload to the AI API. The AI service executes the deep learning inference and returns the diagnostic probability score. This architectural pattern ensures that heavy computational processing does not block the main event loop of the clinic's operational system.

Cloud Architecture and Deployment Strategy

To guarantee high availability and real-time performance, the system is designed for cloud-native deployment. Application components are containerized. This containerization ensures environmental consistency across development, testing, and production phases. The deployment architecture leverages virtual private server (VPS) cloud infrastructure to host these containers.

To handle high-resolution medical images required for AI analysis, the system

integrates a Content Delivery Network (CDN) and Cloud Object Storage. This offloads heavy media files from the core database, accelerating data retrieval times for end-users across different geographical locations.

The comprehensive containerized deployment strategy and the integration with external cloud managed services are depicted in **Figure 3.2**

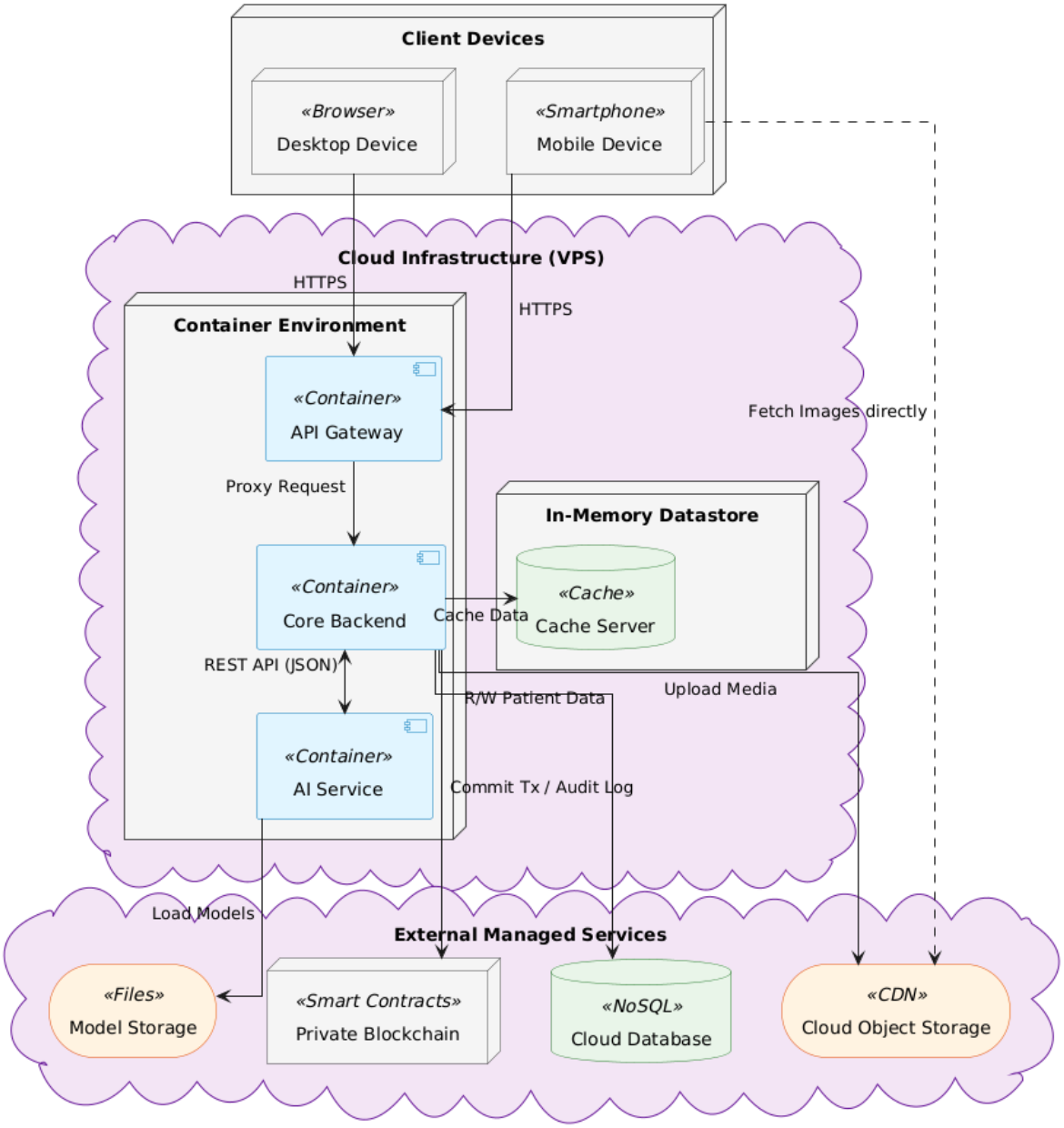

Figure 3.2: Cloud-native Deployment and Containerization Strategy.

Technical Issues and System Solutions

Developing a healthcare platform introduces critical challenges regarding data security and performance. The architecture proactively addresses these through specific technical solutions:

- **Security & Compliance**: Medical data is highly sensitive and requires strict protection. The system employs multiple security layers, including End-to-End encryption via HTTPS/SSL-TLS for data in transit and one-way hashing algorithms for passwords. Unauthorized access is prevented through Stateless Authentication and strict RBAC. Furthermore, Input Sanitization mechanisms

are implemented to neutralize Injection and XSS attacks. Most importantly, the integration of an internal Blockchain network provides a transparent, tamper-proof log of all data interactions.

- **Performance & Scalability**: The system must handle concurrent user requests and intensive AI inference without latency. To resolve this, a Caching Layer is implemented to serve frequently accessed data instantly. For long-running tasks, such as batch AI processing or report generation, the system utilizes Message Broker Queues for asynchronous processing. This ensures the system remains highly responsive, providing a seamless experience for medical staff during peak operational hours.

System Design

System Overview

Traditional clinic systems suffer from operational inefficiencies, lack of diagnostic support, and data security vulnerabilities. To address this, we propose a Smart Clinic System integrating automated management, AI-driven disease prediction (KNN/CNN), and Blockchain for tamper-proof medical records.

Human Actors

User: User is a general actor representing all system users who can access authentication functionalities such as login and account management.

Admin: Oversees complete system management, including user accounts, role assignments, system configurations, report generation, and clinic monitoring, ensuring seamless operations.

Doctor : The Doctor is responsible for diagnosing patients, updating medical records, prescribing treatment, and reviewing AI-based disease prediction results. Doctors are the main clinical decision-makers in the system.

Patient: The Patient can register an account, book or cancel appointments, view medical records, receive treatment results, and interact with doctors through consultation features.

Receptionst: The Receptionist manages patient check-in, schedules appointments, and coordinates patient flow within the clinic to ensure smooth daily operations.

Cashier: The Cashier handles billing processes, generates invoices, manages payments, and ensures financial transactions related to medical services are recorded accurately.

Pharmacy Manager: The Pharmacy Manager manages medicine inventory, updates stock information, and controls the dispensing of medicines based on doctors' prescriptions.

System Actors

AI System: Provides machine learning-based disease prediction, including diabetes prediction (KNN) and skin disease detection (CNN). It returns analytical results to support medical decision-making.

Blockchain System: Ensures secure storage and integrity of medical records by

hashing data and storing it in a distributed ledger, allowing verification and tamper-proof history tracking

Use Case Diagram

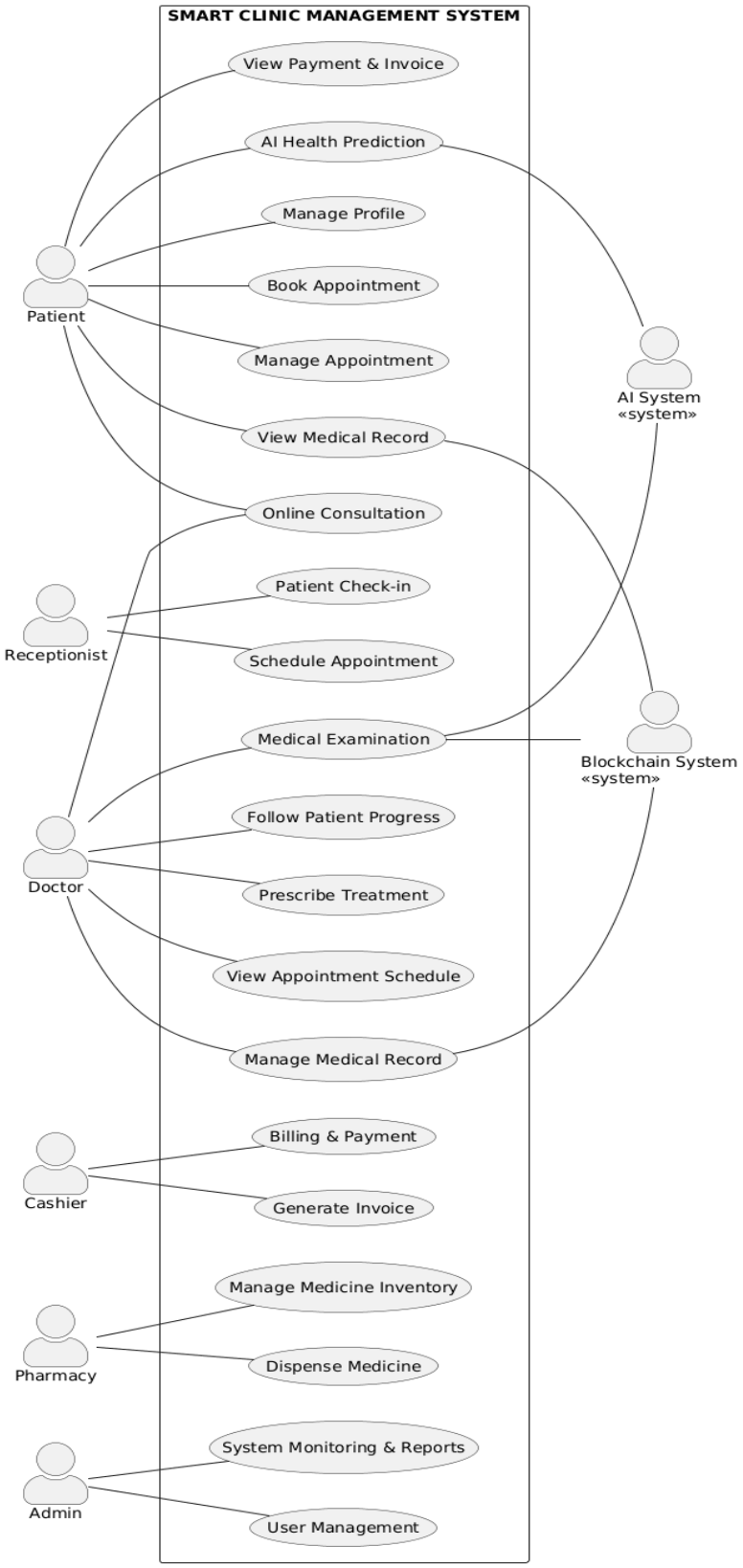

Figure 3.3 Use Case Diagram of Smart Clinic System

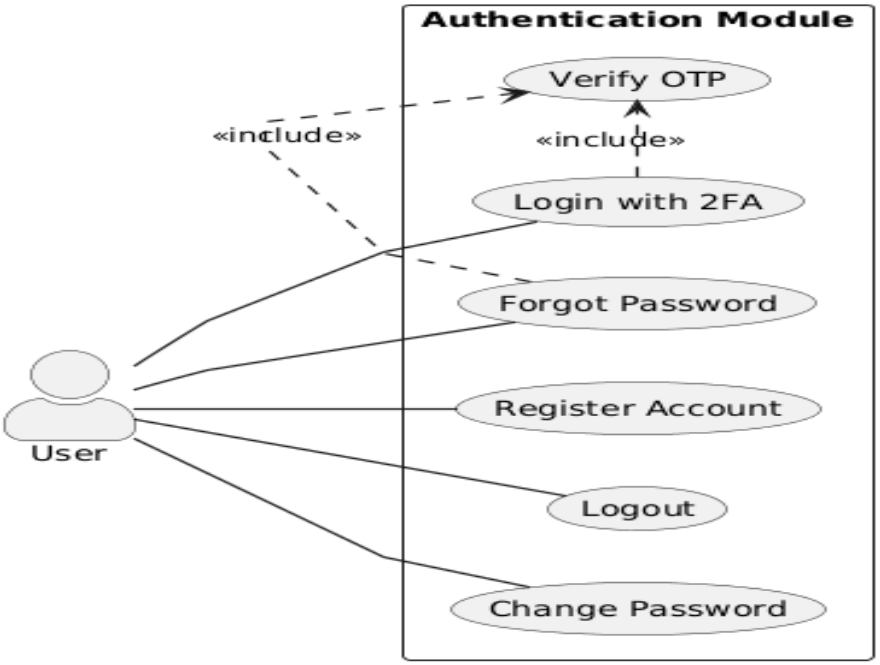

Figure 3.4 Authentication Use Case Diagram

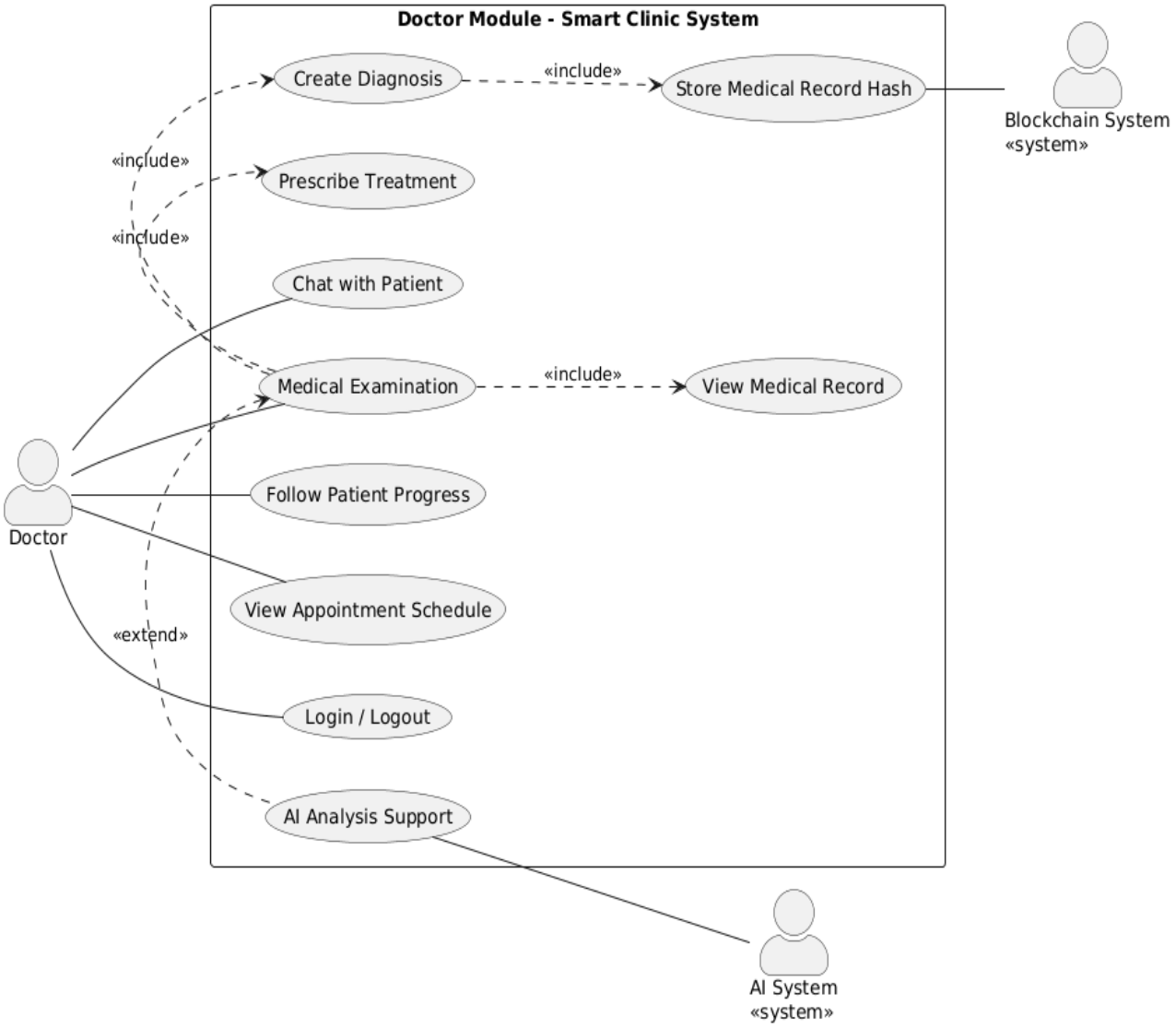

Figure 3.5: Doctor Use Case Diagram

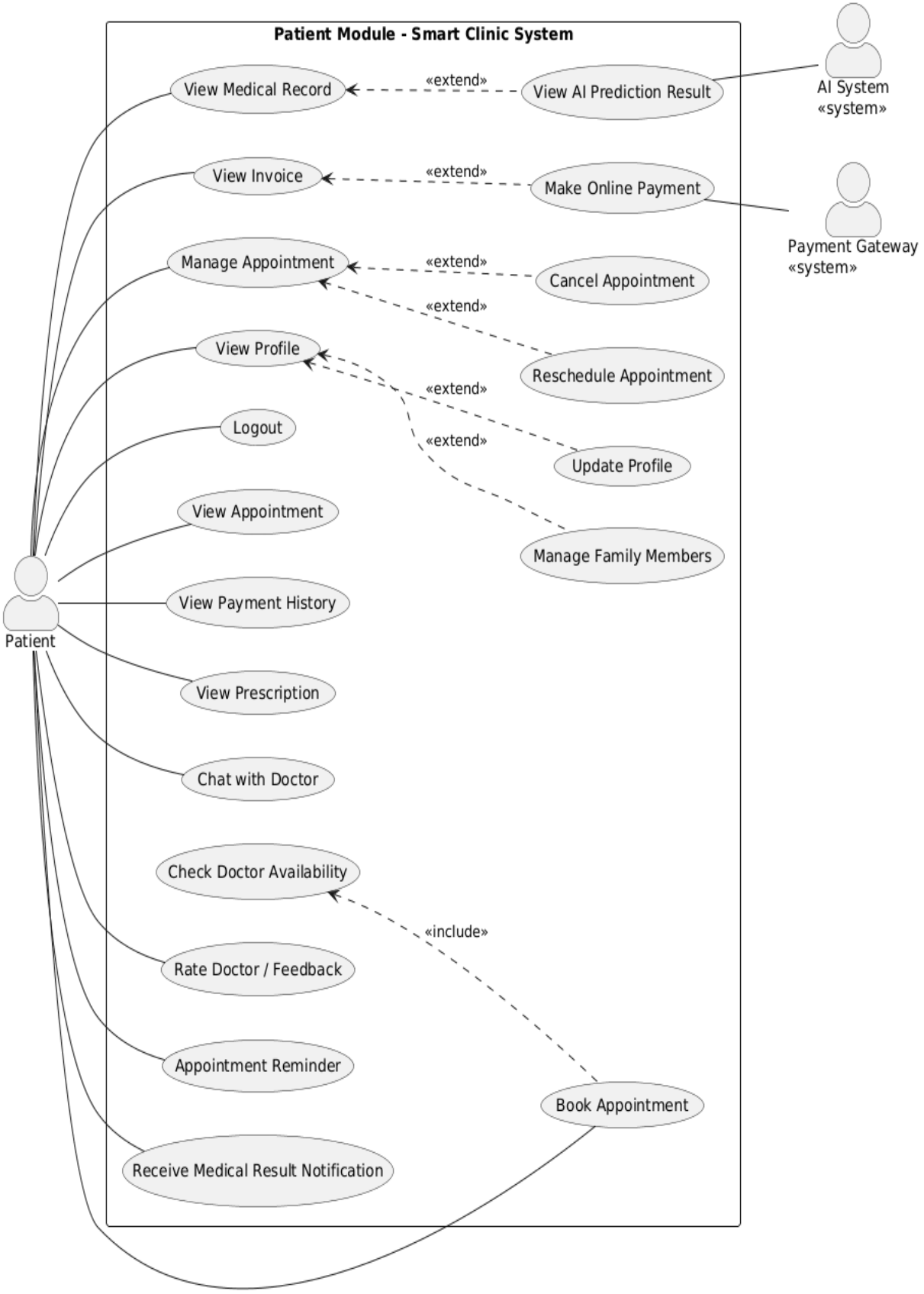

Figure 3.6: Patient Use Case Diagram

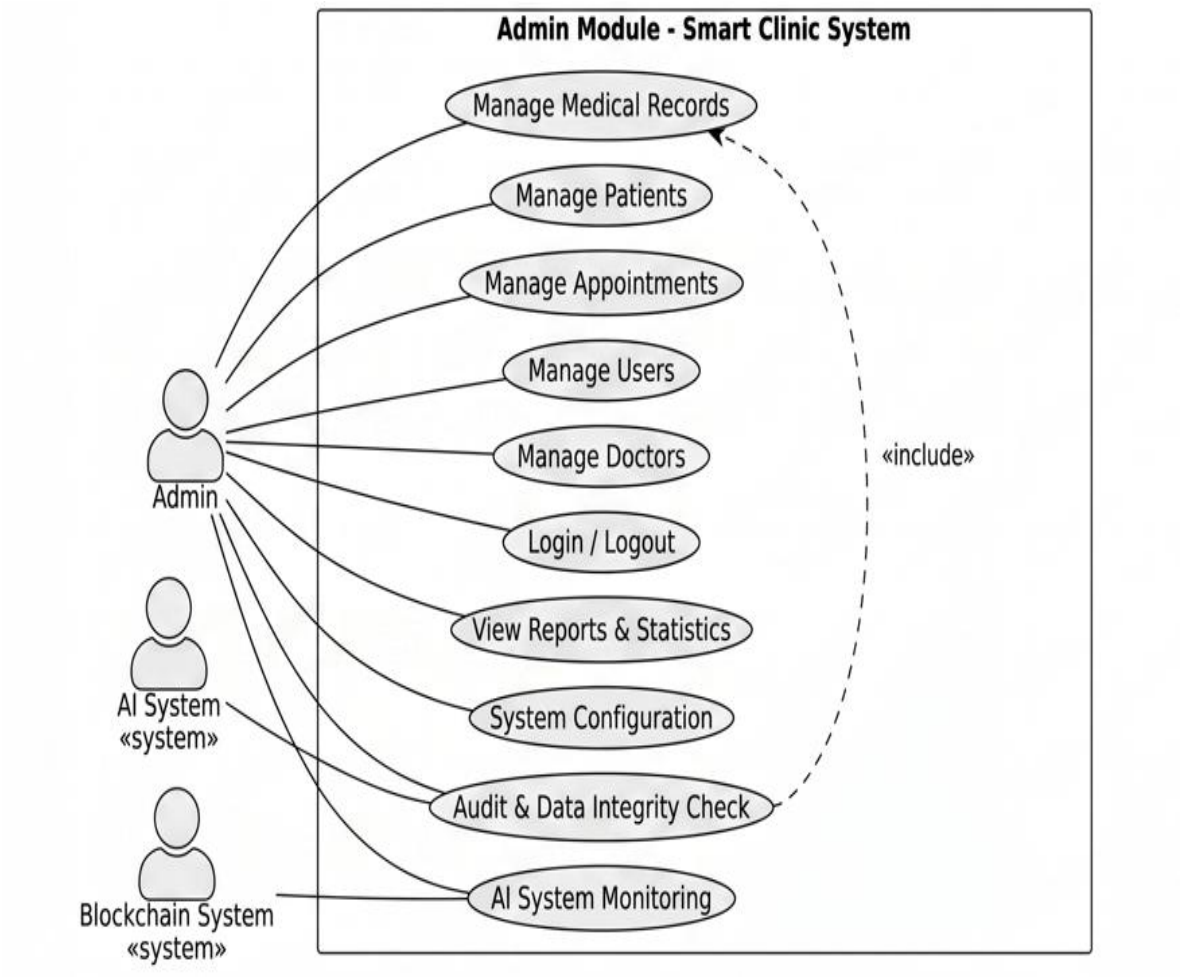

Figure 3.7: Admin Use Case Diagram – Smart Clinic System

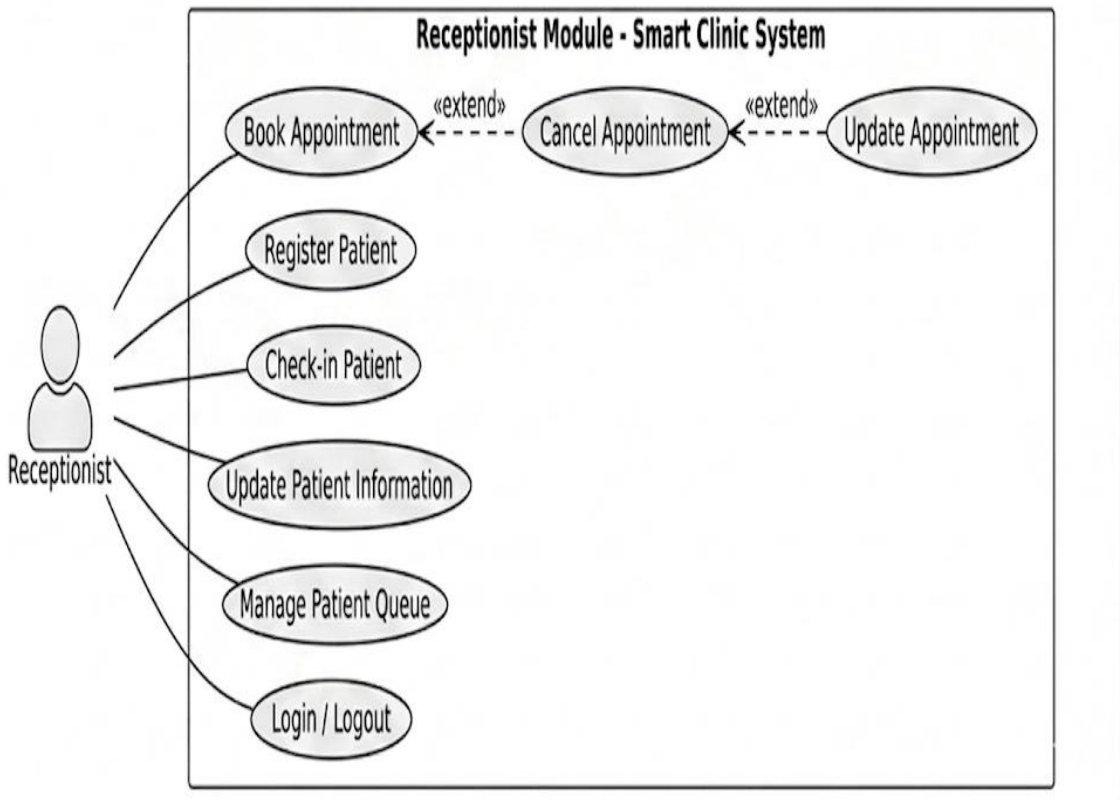

Figure 3.8: Receptionist Use Case Diagram

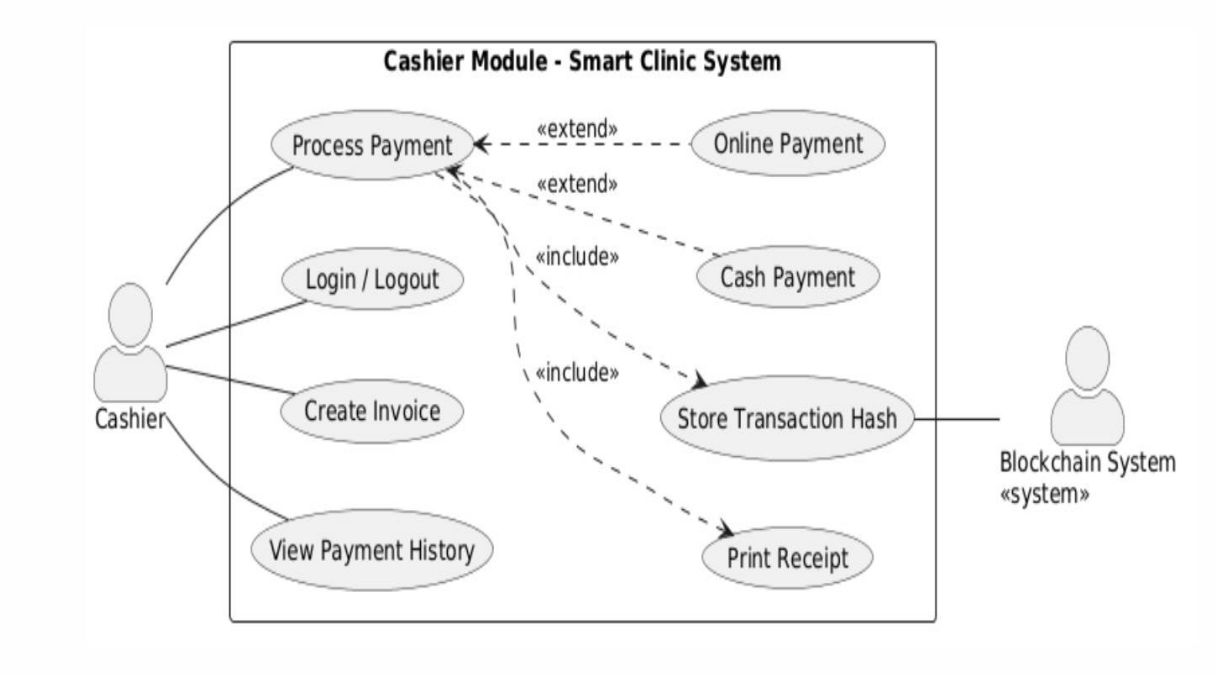

Figure 3.9: Cashier Use Case Diagram

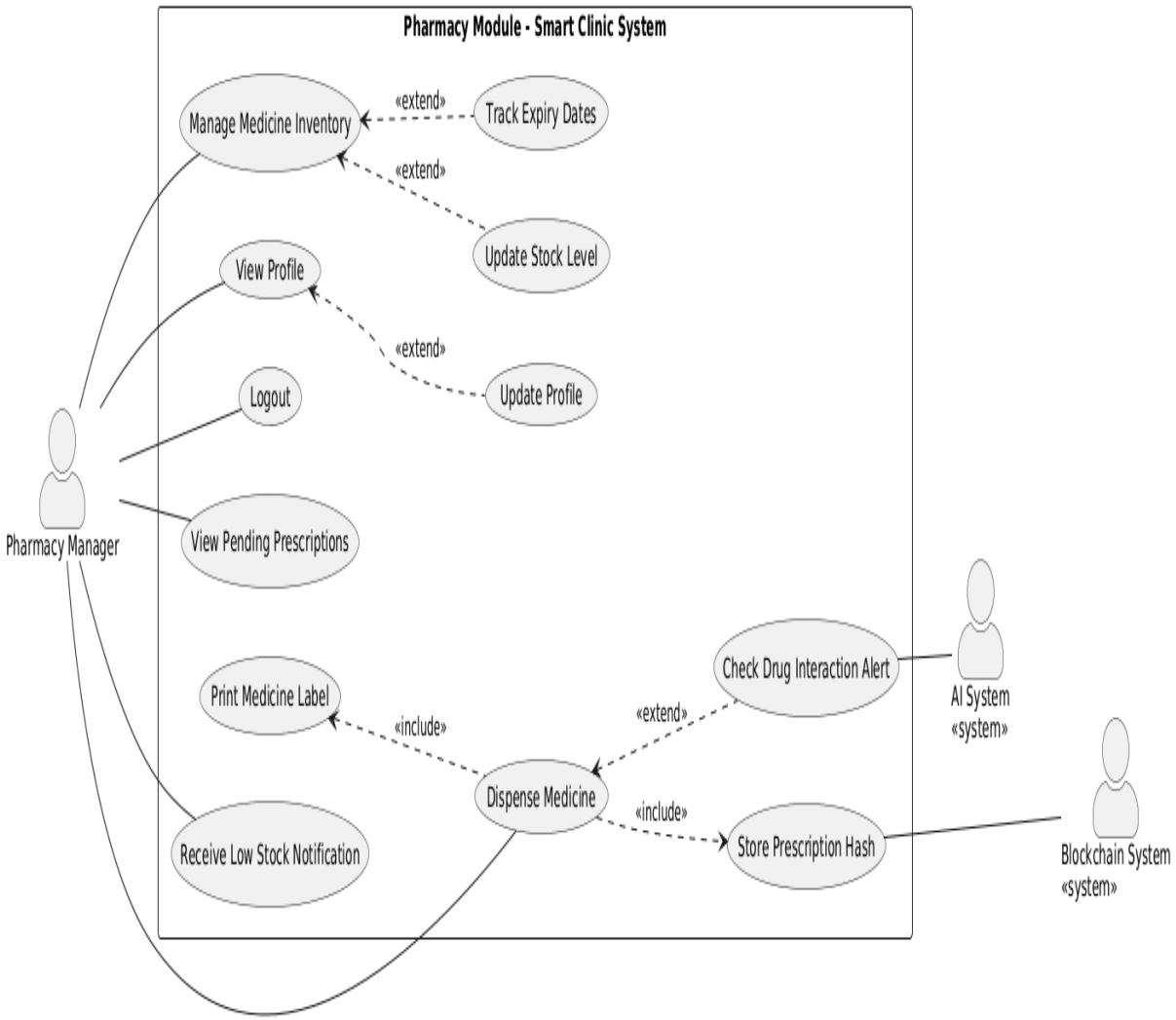

Figure 3.10: Pharmacy Use Case Diagram

Use Case Specification

1. Use Case 1: Booking Apointment

- **Use Case Name**: Book Appointment
- **Actor**: Patient, Receptionist
- **Description**: Enables a patient to select a specialty, choose an available doctor and time slot, and securely book a medical visit. The receptionist then monitors and updates the status of the booking upon arrival.
- **Pre-conditions**: The Patient is logged into the Flutter Mobile application with an active account.
- **Basic Flow:**
  1. The Patient navigates to the "Book Appointment" module.
  2. The system fetches and renders a live list of medical specialties and available doctors from the database.
  3. The Patient selects a doctor, an available date, and a specific time slot.
  4. The Patient inputs primary symptoms and confirms the booking request.
  5. The system validates slot availability, reserves the slot, generates a unique Booking ID, and marks the status as "Pending".
  6. The system sends a real-time notification to the Patient and updates the Receptionist's dashboard queue.
  7. Upon physical arrival at the clinic, the Receptionist verifies the Booking ID and updates the status to "Checked-In".
- **Alternative Flows:**
  - 5a. Slot Conflict: If another user completes a booking for the same slot simultaneously, the system cancels the current transaction, alerts the patient, and requests them to select another time.
  - 7a. Patient Cancellation: The Patient can trigger a cancellation request through the app before the scheduled time; the system frees up the slot and archives the appointment status as "Cancelled".

Figure 3.11: Activity Diagram for Booking Appointment (UC1)

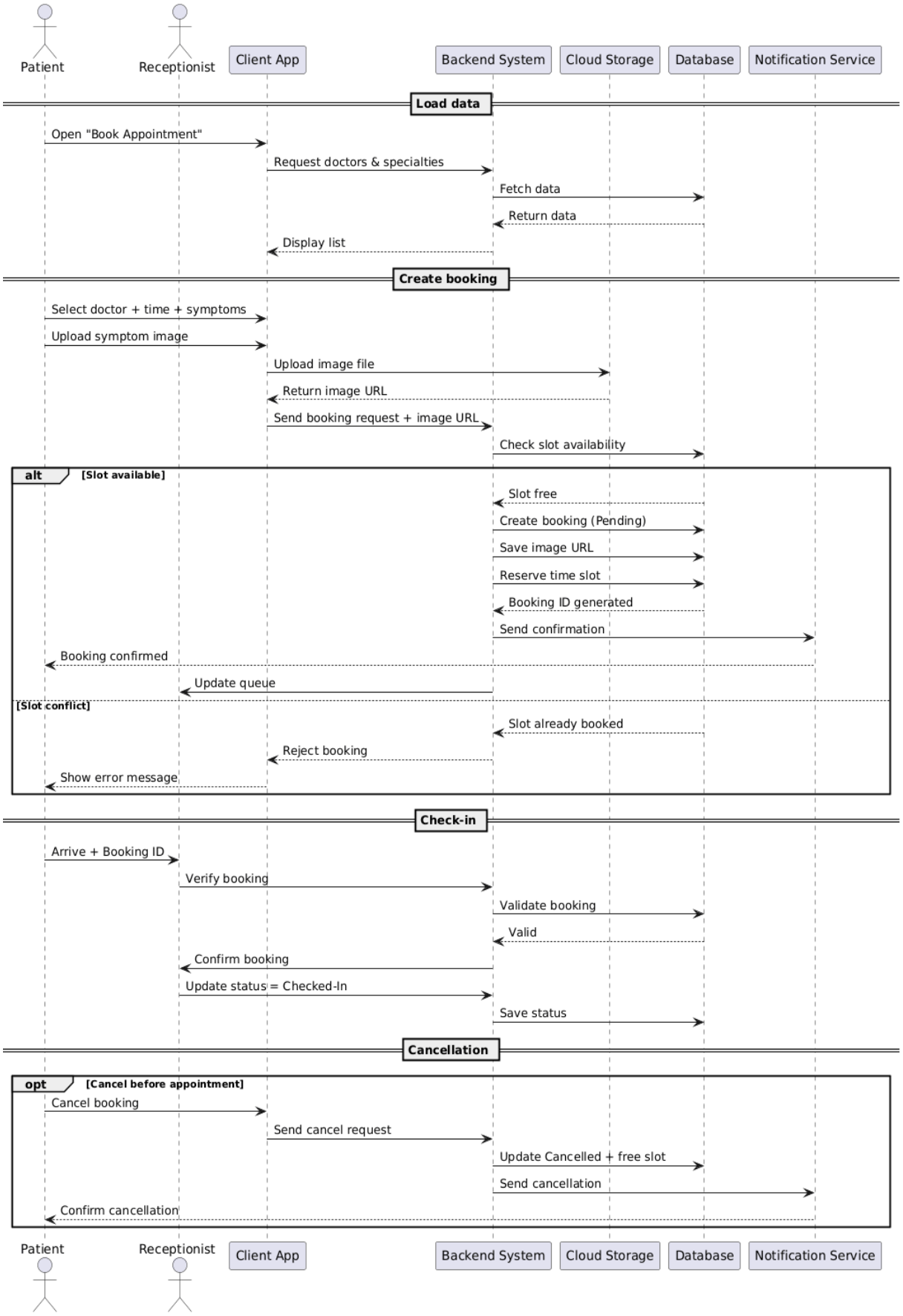

Figure 3.12 Sequence Diagram for Booking Apointment (UC1)

2. Use Case 2: Medical Examination

- **Use Case Name**: Medical Examination
- **Actor**: Doctor, AI Service
- **Description**: Enables the attending doctor to review a patient's clinical history, input exam metrics, and leverage an independent deep learning inference microservice to analyze skin lesion images (Acne vs. Skin Cancer) or predict Diabetes risks. The flow ends when the AI's predictive outcome is rendered on the doctor's screen for clinical review.
- **Pre-conditions:** The patient holds a status of "Checked-In" within the consultation queue. The doctor is securely authenticated into the Web Dashboard.
- **Basic Flow:**
  1. The Doctor selects the next patient from the live digital queue on the dashboard.
  2. The system retrieves the patient's longitudinal medical history from the database and renders it.
  3. The Doctor conducts the physical examination and logs current clinical metrics.
  4. The Doctor uploads the specialized clinical input based on the diagnostic context:
  5. For Dermatological Analysis: Uploads a high-resolution image of the patient's skin lesion.
  6. For Diabetes Prediction: Inputs a structured matrix of physiological indicators (Glucose, HbA1c, BMI, and Age).
  7. The Doctor triggers the "Execute AI Analysis" request.
  8. The Core Backend processes the payload into a structured JSON transmission package (converting skin images into Base64 format or structuring physiological fields into numerical arrays).
  9. The Core Backend transmits an asynchronous HTTP POST request to the isolated Flask AI Microservice.
  10. The AI Service routes the payload to the corresponding pre-trained model component (CNN for skin lesions or Machine Learning Classifiers for diabetes) to execute the deep learning inference pipeline.
  11. The AI Service returns a standardized JSON response containing the prediction outcome and its mathematical Confidence Interval (Probability Score) to the Core Backend.
  12. The system relays the AI diagnostic recommendations onto the Doctor's active screen in real-time, allowing the doctor to review the intelligent insights before clinical decision-making.
- **Alternative Flows:**
  - 4a. Incompatible Payload Format: If the asset violates constraints

(corrupted image or missing glucose fields), the system aborts the request, displays a validation error, and prompts for re-input.

- 8a. AI Microservice Timeout/Failure: If the AI service fails to respond within 5 seconds, the system bypasses model execution, logs a warning, alerts the doctor that AI is offline, and enables manual diagnostic entry to prevent clinical delays.

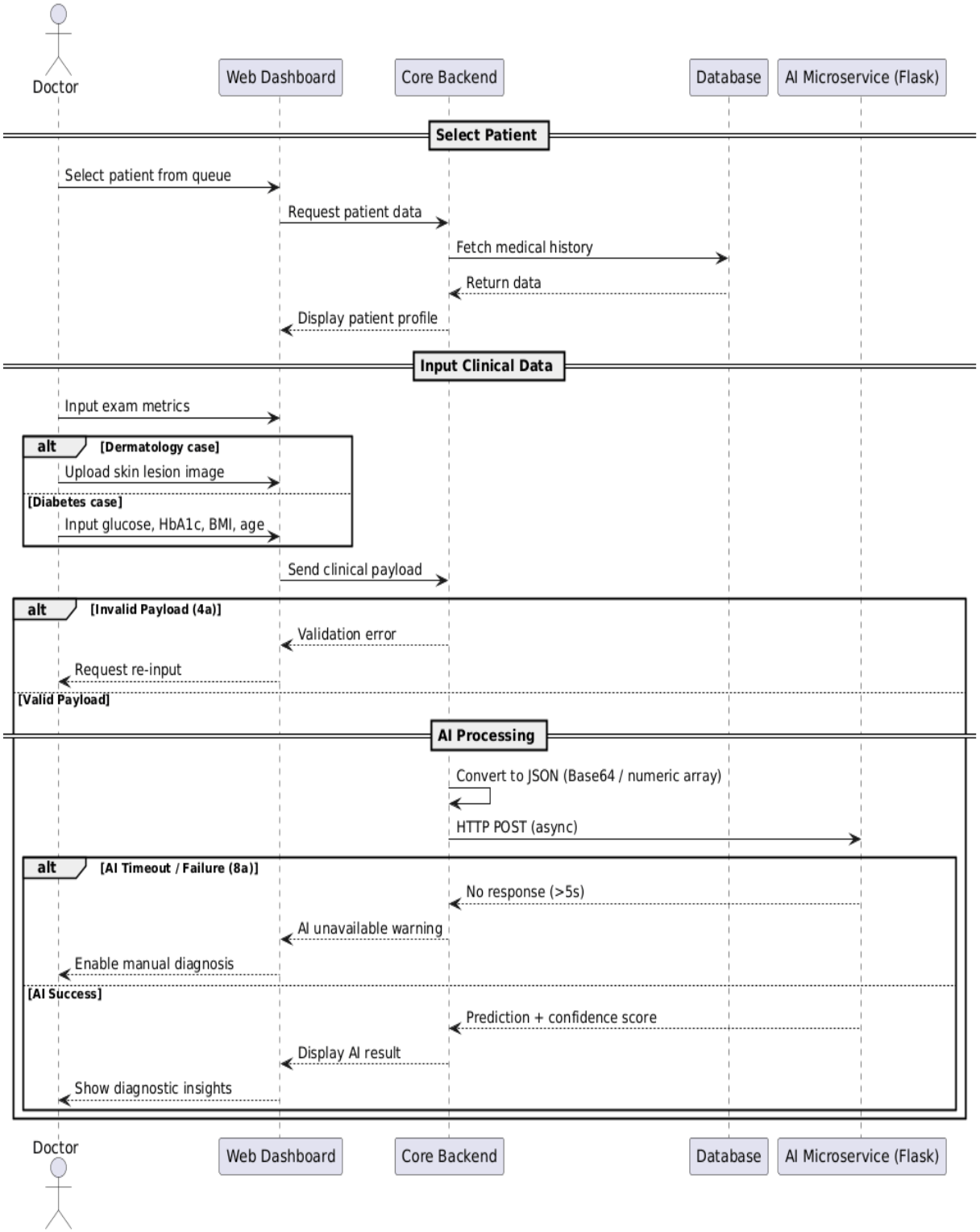

Figure 3.11 Sequence Diagram for Medical Examination with AI Support

3.Use Case 3: Store Medical Record Hash

- **Use Case Name:** Store Medical Record Hash
- **Actor:** Core Backend System, Blockchain Network (Hyperledger Fabric)
- **Description:** Acts as the ultimate security checkpoint of the system. Upon the finalization of a medical examination, the Core Backend automatically computes a cryptographic SHA-256 hash of the clinical data and invokes a smart contract to anchor this hash onto the distributed ledger, ensuring the absolute immutability and auditability of the electronic medical record (EMR).
- **Pre-conditions:** The doctor has successfully finalized the diagnosis and prescription in the previous workflow (Use Case 2). The Core Backend has a valid gRPC connection established with the Hyperledger Fabric network nodes.
- **Basic Flow:**
  1. The Doctor clicks the "Finalize Medical Record & Submit" button on the clinical dashboard.
  2. The Core Backend intercepts the finalized EMR payload and serializes the entire dataset (including patient metrics, AI metadata, and prescription details) into a canonical JSON format.
  3. The Core Backend executes a SHA-256 cryptographic hashing algorithm on the serialized data to generate a unique 64-character string (EMR_Hash).
  4. The Core Backend invokes the Hyperledger Fabric Smart Contract (Chaincode) via a secure gRPC channel, transmitting the Record_ID and the generated EMR_Hash to the network's Endorsing Peers.
  5. The Blockchain network validates the transaction, executes the smart contract logic, achieves consensus via the Ordering Service, and permanently appends the transaction block to the ledger.
  6. The Hyperledger Fabric network returns a transaction success receipt containing a unique cryptographic Transaction Hash (Tx_Hash).
  7. Upon receiving the Tx_Hash, the Core Backend writes the raw medical record into the off-chain MongoDB database, explicitly embedding the Tx_Hash within the document as an immutable cross-reference anchor.
  8. The system updates the UI, notifying the doctor that the medical record has been successfully secured and anchored.
- **Alternative Flows:**
  - 4a. Blockchain Network Disconnection: If the Hyperledger Fabric network is unreachable during the Smart Contract invocation, the system prevents clinical workflow blockage by saving the EMR to MongoDB with a temporary status of "Pending Ledger Sync". An automated background cron job is queued to re-attempt the cryptographic commit once the ledger network recovers.

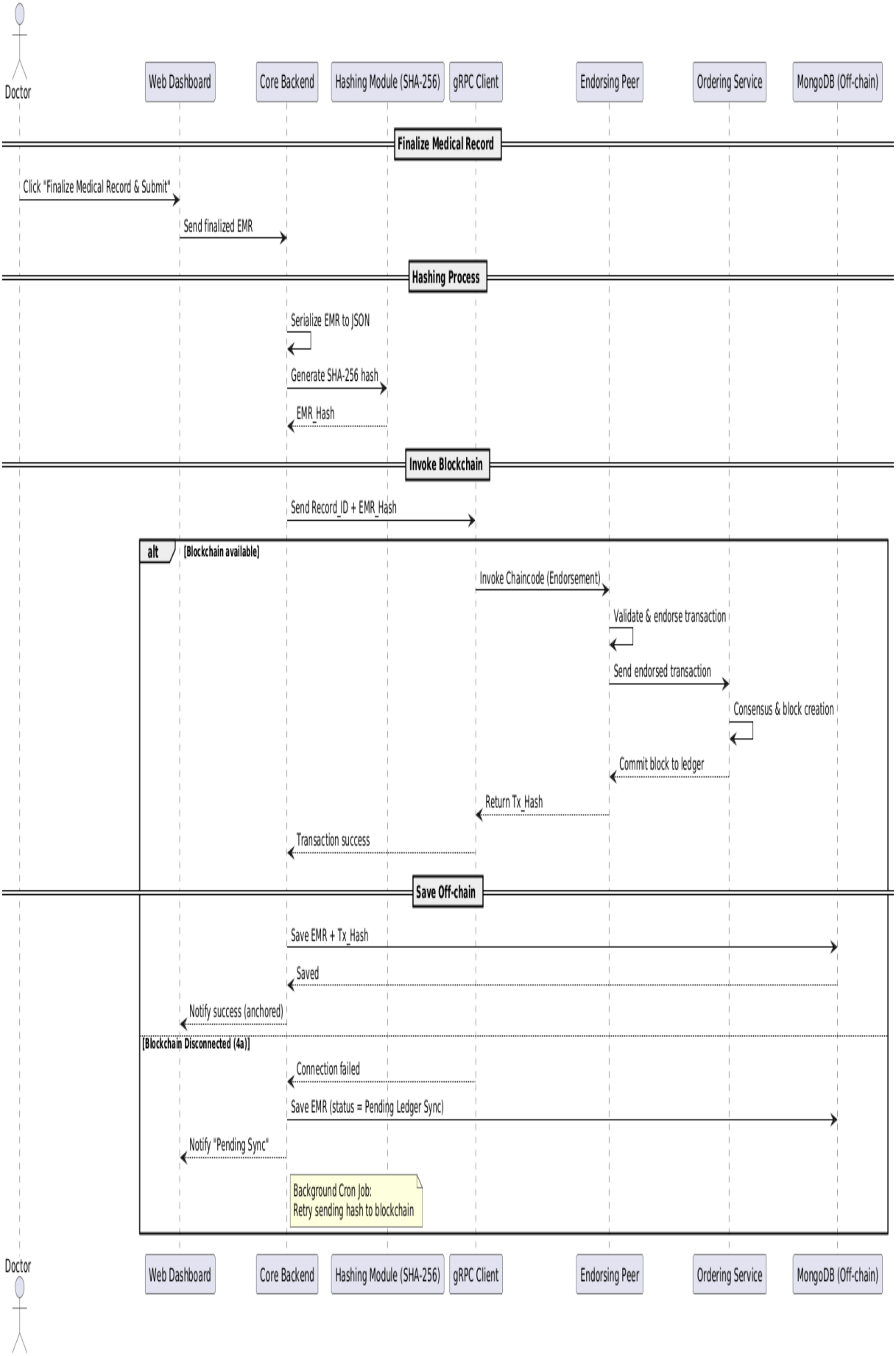
Doctor
Web Dashboard
Core Backend
Hashing Module (SHA-256)
gRPC Client
Endorsing Peer
Ordering Service
MongoDB (Off-chain)
Finalize Medical Record
Click "Finalize Medical Record & Submit"
Send finalized EMR
Hashing Process
Serialize EMR to JSON
Generate SHA-256 hash
EMR_Hash
Invoke Blockchain
Send Record_ID + EMR_Hash
alt
[Blockchain available]
Invoke Chaincode (Endorsement)
Validate & endorse transaction
Send endorsed transaction
Consensus & block creation
Commit block to ledger
Return Tx_Hash
Transaction success
Save Off-chain
Save EMR + Tx_Hash
Saved
Notify success (anchored)
[Blockchain Disconnected (4a)]
Connection failed
Save EMR (status = Pending Ledger Sync)
Notify "Pending Sync"
Background Cron Job:
Retry sending hash to blockchain
Doctor
Web Dashboard
Core Backend
Hashing Module (SHA-256)
gRPC Client
Endorsing Peer
Ordering Service
MongoDB (Off-chain)

Figure 3.12 Sequence Diagram for Store and Verify Medical Record Hash

4. Use Case 4: Dispense Medicine

- **Use Case Name:** Dispense Medicine
- **Actor:** Pharmacy Manager
- **Description:** Governs the final step of the clinical workflow. It ensures the Pharmacy Manager dispenses verified medications based on "Paid" digital prescriptions, automatically cross-checks expiration dates, executes atomic stock decrements in the database, and flags inventory alerts.
- **Pre-conditions:** The patient has completed the billing process, and the digital prescription status is marked as "Paid" or "Pending Dispense".
- **Basic Flow:**
  1. The Pharmacy Manager accesses the active queue and selects a "Paid" prescription.
  2. The system scans the database to verify the expiration dates and physical quantities of the prescribed medications in real-time.
  3. The Pharmacy Manager prepares the physical medications and prints dosage instruction labels.
  4. The Pharmacy Manager physically dispenses the medications to the patient and clicks "Confirm Dispense" on the dashboard.
  5. The system triggers an **atomic database transaction** to decrement the Stock_Level of each dispensed item.
  6. The system verifies the remaining stock against a predefined minimum threshold. If the stock is adequate, it proceeds without alerts.
  7. The system updates the prescription status to "Dispensed" and archives the transaction.
- **Alternative Flows:**
  - 2a. Inventory Exception (Out of Stock / Expired): If the system detects that any prescribed drug has expired or lacks sufficient physical quantity, it immediately blocks the dispensing flow, displays a high-severity "Inventory Exception" alert, and prompts the manager to notify the prescribing doctor for a prescription modification.
  - 6a. Low Stock Warning: If the atomic decrement causes the remaining stock to fall below the minimum safety threshold, the system silently generates a "Low Stock Notification" and dispatches it to the Administrator's dashboard for restocking.

Figure 3.13 Activity Diagram for Dispense Medicine with Stock Update

5. Use Case 5: Audit & Data Integrity Check

- **Use Case Name**: Audit & Data Integrity Check
- **Actor**: System Administrator, Blockchain Network
- **Description**: Provides an automated cryptographic audit mechanism. It recalculates the SHA-256 hash of a medical record stored in the off-chain MongoDB and compares it against the original immutable hash anchored on the decentralized ledger to detect unauthorized data tampering.
- **Pre-conditions**: The medical record is fully saved in the database with an associated Tx_Hash referencing the blockchain transaction.
- **Basic Flow:**
  1. The Administrator navigates to the Audit Dashboard and selects a specific

Electronic Medical Record (EMR) to initiate the integrity check.

2. The Core Backend queries the MongoDB database to fetch the target EMR document (including the embedded Tx_Hash).
3. The Core Backend dynamically executes the SHA-256 algorithm on the retrieved clinical data to generate a fresh cryptographic hash (Current_Hash).
4. The Core Backend invokes the Hyperledger Fabric ledger via gRPC, using the Tx_Hash to retrieve the original anchored hash (Ledger_Hash) generated at the time of examination.
5. The Core Backend performs a strict mathematical comparison between the Current_Hash and the Ledger_Hash.
6. Since the hashes match identically, the system confirms data integrity, logs a successful audit event, and renders a green "Integrity Verified - No Tampering Detected" status on the dashboard.

- **Alternative Flows:**
    - 5a. Tampering Detected (Hash Mismatch): If the Current_Hash differs from the Ledger_Hash, it mathematically proves the off-chain database has been compromised or illicitly modified. The system instantly aborts the verification, triggers a critical security lockdown on the record, flags it as "COMPROMISED", and dispatches an emergency alert to the administrative team for forensic investigation.

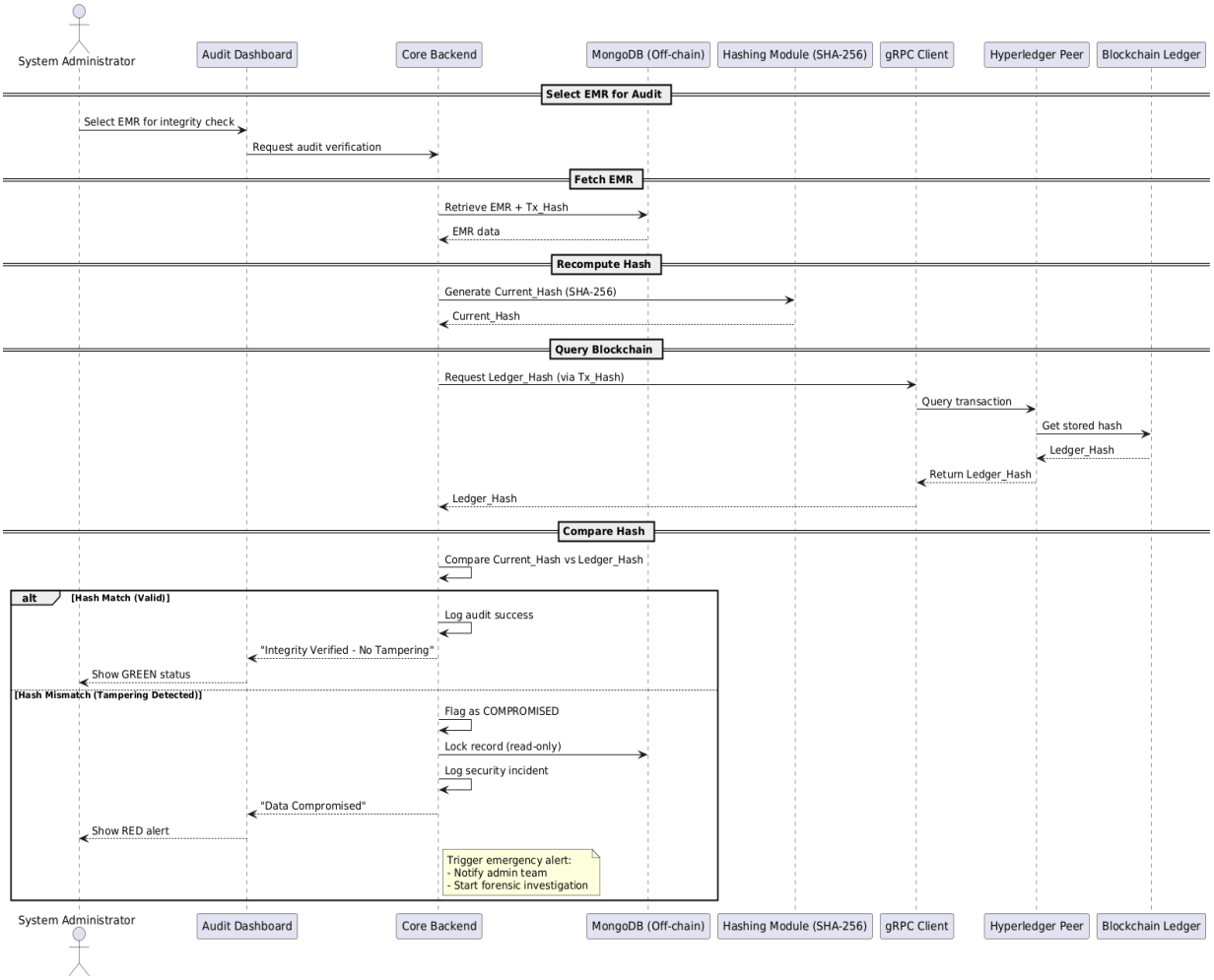

Figure 3.14 Sequence Diagram for Audit & Data Integrity Check

Database Design

Database Architecture Selection

This system adopts a NoSQL architecture (MongoDB Atlas) over traditional relational databases (SQL) to address two core technical requirements. First, MongoDB's dynamic schema provides the flexibility needed to natively store diverse and unstructured metadata generated by different AI models (CNN for dermatology, KNN for diabetes) without rigid table definitions. Second, the Embedded Document model allows the system to encapsulate an entire medical encounter—including clinical metrics, AI predictions, and prescriptions—into a single, cohesive JSON object. This structure eliminates the need for complex JOIN operations, making it highly efficient to compute a unified SHA-256 hash of the record for secure, immutable anchoring on the Blockchain.

Collection Relationship Diagram

The diagram below illustrates the overall data model of the system based on the NoSQL architecture. Unlike the traditional Entity-Relationship Diagram (ERD) used in relational databases, this diagram serves as a comprehensive architectural map, demonstrating how Collections interact with each other through MongoDB's two core mechanisms: document referencing (<<ref>>) and document embedding

(<<embedded>>).

Through the cardinality indicated at the ends of the relationship arrows, the diagram clearly defines one-to-one (1 Medical Record generates 1 Invoice) and one-to-many relationships (1 User can have multiple Appointments; 1 Prescription references multiple Inventory items). This structure ensures that data across various modules (Clinical, Artificial Intelligence, Billing, and Pharmacy) is tightly integrated, thereby optimizing retrieval performance and guaranteeing transaction atomicity within the system.

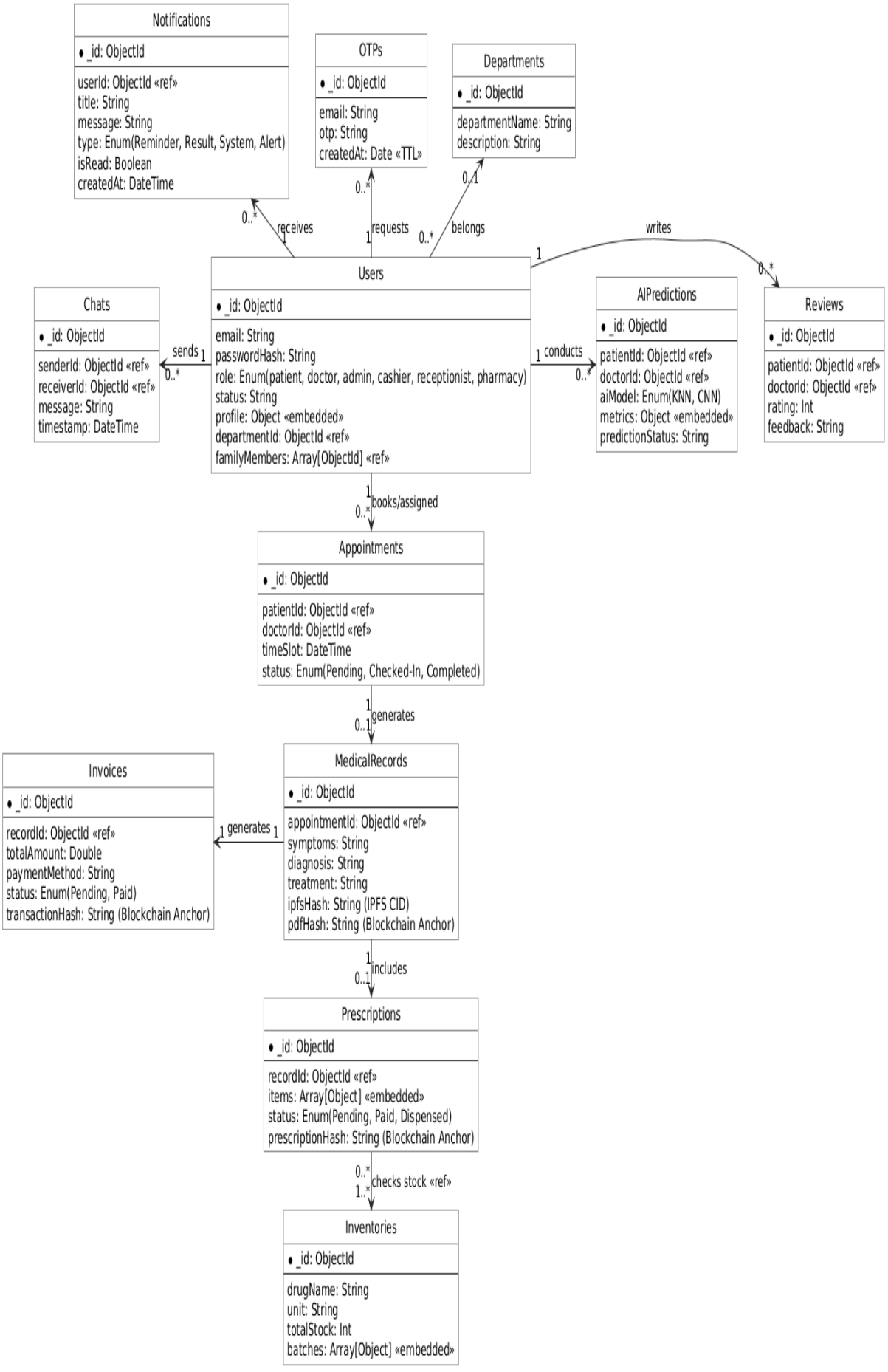
Notifications
_id: ObjectId
userId: ObjectId «ref»
title: String
message: String
type: Enum(Reminder, Result, System, Alert)
isRead: Boolean
createdAt: DateTime
OTPs
_id: ObjectId
email: String
otp: String
createdAt: Date «TTL»
Departments
_id: ObjectId
departmentName: String
description: String
receives
requests
belongs
writes
Users
_id: ObjectId
email: String
passwordHash: String
role: Enum(patient, doctor, admin, cashier, receptionist, pharmacy)
status: String
profile: Object «embedded»
departmentId: ObjectId «ref»
familyMembers: Array[ObjectId] «ref»
Chats
_id: ObjectId
senderId: ObjectId «ref»
receiverId: ObjectId «ref»
message: String
timestamp: DateTime
sends
AIPredictions
_id: ObjectId
patientId: ObjectId «ref»
doctorId: ObjectId «ref»
aiModel: Enum(KNN, CNN)
metrics: Object «embedded»
predictionStatus: String
conducts
Reviews
_id: ObjectId
patientId: ObjectId «ref»
doctorId: ObjectId «ref»
rating: Int
feedback: String
books/assigned
Appointments
_id: ObjectId
patientId: ObjectId «ref»
doctorId: ObjectId «ref»
timeSlot: DateTime
status: Enum(Pending, Checked-In, Completed)
generates
Invoices
_id: ObjectId
recordId: ObjectId «ref»
totalAmount: Double
paymentMethod: String
status: Enum(Pending, Paid)
transactionHash: String (Blockchain Anchor)
generates
MedicalRecords
_id: ObjectId
appointmentId: ObjectId «ref»
symptoms: String
diagnosis: String
treatment: String
ipfsHash: String (IPFS CID)
pdfHash: String (Blockchain Anchor)
includes
Prescriptions
_id: ObjectId
recordId: ObjectId «ref»
items: Array[Object] «embedded»
status: Enum(Pending, Paid, Dispensed)
prescriptionHash: String (Blockchain Anchor)
checks stock «ref»
Inventories
_id: ObjectId
drugName: String
unit: String
totalStock: Int
batches: Array[Object] «embedded»

Figure 3.15: NoSQL Document Model Diagram.

3.6.3 Core Collections Schema Definition

This section defines the structural schema for the core collections within the MongoDB database. The design leverages NoSQL's flexible schema capabilities, utilizing embedded documents for high-performance read operations and explicit references for relational mapping.

1. Users Collection

- **Description:** A polymorphic collection managing all system actors (Patient, Doctor, Admin, etc.). It embeds dynamic personal information within the `profile` object to optimize data retrieval and prevent schema sparsity.
- **JSON Schema:**

```
{
  "_id": "ObjectId",
  "email": "String (Unique)",
  "passwordHash": "String (Bcrypt Hashed)",
  "role": "String (Enum: patient, doctor, admin, cashier, receptionist, pharmacy)",
  "status": "String (active, inactive)",
  "profile": {
    "phone": "String",
    "avatar": "String (Cloudinary URL)",
    "gender": "String",
    "address": "String",
    "// Note": "Dynamic fields like 'specialty' (Doctor) or 'allergies' (Patient) are dynamically embedded here based on the role."
  },
  "departmentId": "ObjectId (Ref: Departments - Nullable for non-doctors)",
  "familyMembers": ["ObjectId (Ref: Users)"],
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

2. Appointments Collection

- **Description:** Manages medical bookings and schedules. This schema deliberately utilizes data denormalization by embedding critical patient details (patientName, medicalHistory, allergies) directly into the appointment document. This NoSQL optimization completely eliminates the need for expensive $lookup queries, allowing the Doctor's dashboard to load the daily consultation queue in O(1) query time. It also supports pre-examination visual context via the imageUrl field (crucial for dermatology).

- **JSON Schema:**


```
{
 "_id": "ObjectId",
 "patient": "ObjectId (Ref: Users)",
 "doctor": "ObjectId (Ref: Users)",

 "// Denormalized Patient Context": "Optimized for fast dashboard rendering",
 "patientName": "String",
 "phone": "String",
 "gender": "String",
 "address": "String",
 "medicalHistory": "String",
 "allergies": "String",

 "// Booking Details": "",
 "reason": "String (e.g., 'vùng da mặt xuất hiện nhiều mụn')",
 "date": "String (Format: YYYY-MM-DD)",
 "time": "String (Format: HH:MM AM/PM)",
 "imageUrl": "String (Cloudinary URL - Pre-examination asset)",
 "status": "String (Enum: pending, completed, cancelled)",
 "createdAt": "ISODate",
 "updatedAt": "ISODate"
}
```


3. MedicalRecords Collection

- **Description:** The central clinical entity. It implements a Hybrid Storage architecture. Large clinical files are stored off-chain on IPFS, while cryptographic hashes are stored on-chain to guarantee data integrity.
- **JSON Schema:**


```
{
 "_id": "ObjectId",
 "appointmentId": "ObjectId (Ref: Appointments)",
 "symptoms": "String",
 "diagnosis": "String",
 "treatment": "String",
```

```
 "pdfUrl": "String (Link tải PDF)",
 "ipfsHash": "String (CID từ IPFS)",
 "pdfHash": "String (SHA-256 để verify)",
 "createdAt": "ISODate"
}
```

4. AIPredictions Collection

- **Description:** Stores the inputs and outputs of the isolated AI Microservice. The metrics object is schema-less, adapting its structure based on the specific AI model executed.
- **JSON Schema:**

```
{
 "_id": "ObjectId",
 "patientId": "ObjectId (Ref: Users)",
 "doctorId": "ObjectId (Ref: Users)",
 "aiModel": "String (Enum: KNN_Diabetes, CNN_Dermatology)",
 "metrics": {
  "// For KNN_Diabetes": "urea (Double), hba1c (Double), bmi (Double), etc.",
  "// For CNN_Dermatology": "imageUrl (String)"
 },
 "predictionStatus": "String (e.g., High Risk, Low Risk, Positive, Negative)",
 "createdAt": "ISODate"
}
```

6. Prescriptions Collection

- **Description:** Manages medication dispensing. It embeds an array of prescribed items and maintains a prescriptionHash for auditing pharmacy transactions on the Blockchain.
- **JSON Schema:**

```
{
 "_id": "ObjectId",
 "recordId": "ObjectId (Ref: MedicalRecords)",
 "items": [
  {
   "drugId": "ObjectId (Ref: Inventories)",
   "dosage": "String",
```

```
    "quantity": "Int"
   }
  ],
  "status": "String (Enum: Pending, Paid, Dispensed)",
  "prescriptionHash": "String (SHA-256 Blockchain Anchor)",
  "createdAt": "ISODate"
}
```

5. Inventories Collection

- **Description:** Governs pharmacy stock levels. It embeds a batches array to support First-Expired-First-Out (FEFO) inventory tracking mechanisms.
- **JSON Schema:**

```
JSON
{
  "_id": "ObjectId",
  "drugName": "String",
  "unit": "String",
  "totalStock": "Int",
  "batches": [
   {
    "batchId": "String",
    "quantity": "Int",
    "expiryDate": "ISODate"
   }
  ],
  "updatedAt": "ISODate"
}
```

7. Invoices Collection

- **Description:** Handles billing and financial operations. Includes a transactionHash to ensure the financial records are tamper-proof and cryptographically verifiable.
- **JSON Schema:**

```
{
  "_id": "ObjectId",
  "recordId": "ObjectId (Ref: MedicalRecords)",
```

"totalAmount": "Double",
"paymentMethod": "String (e.g., Cash, Credit Card, E-Wallet)",
"status": "String (Enum: Pending, Paid)",
"transactionHash": "String (Blockchain Anchor for financial audit)",
"createdAt": "ISODate",
"paidAt": "ISODate"
}

Class Diagram

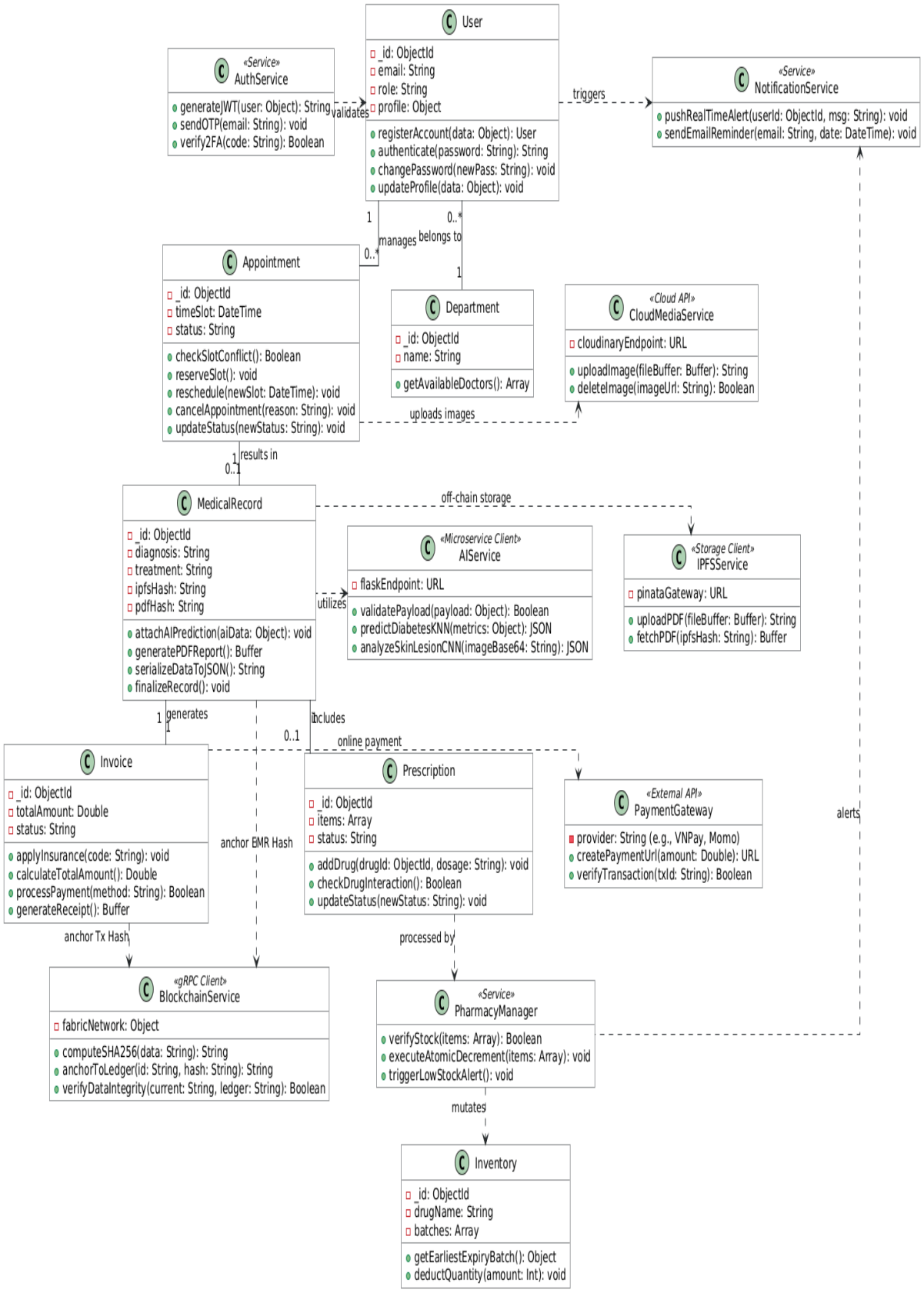

Figure 3.16: Class Diagram.

User Interface and User Experience (UI/UX) Design

The core objective in designing the UI/UX for the Smart Clinic system is to

deliver an intuitive and highly accessible experience for a diverse user base—ranging from elderly patients to busy medical professionals. Rather than applying a fragmented approach, the system strictly adopts a User-Centered Design methodology.

Design System and Visual Guidelines

To guarantee high consistency across both the Mobile Application and the Web Dashboard, the system adheres to the following core design principles:

- Color Palette: The system employs Primary Blue combined with ample White Space as the dominant color scheme. In UI/UX psychology, blue represents trust, safety, and professionalism—standard visual cues in the healthcare industry. Alert colors (Red/Orange) are exclusively utilized when the AI detects high-risk medical conditions or when the inventory system flags low medication stock.
- Typography: The application utilizes sans-serif typefaces (Roboto/Inter). The selection of clean, legible fonts combined with generous line spacing significantly enhances readability, specifically accommodating elderly patients with diminished visual acuity.
- Layout Strategy:
- For the Mobile App (Patients): Implements a Bottom Navigation Bar pattern, allowing users to effortlessly navigate primary modules using a single thumb.
- For the Web Dashboard (Doctors/Admins): Utilizes a Left Sidebar Menu combined with a Grid System. This specific layout optimizes the screen real estate for data-dense interfaces, such as clinical electronic medical records (EMR) and high-resolution AI analysis imagery, eliminating unnecessary scrolling.

2. User Navigation Flow

To minimize the users' cognitive load, business workflows are designed to require the absolute minimum number of interactions (clicks). The state diagram below illustrates the core navigation flow for patients on the Mobile App, covering the two primary functionalities: Appointment Booking and AI Health Prediction.

he application architecture centers around the Home Dashboard, which acts as a primary navigational hub routing users into three independent, closed-loop workflows:

1. AI Prediction Flow: Patients can upload dermatological images or input clinical metrics to receive instant, AI-driven diagnostic insights (CNN/KNN) before returning to the main screen.

2. Appointment Booking Flow: A sequential process allowing users to select a medical specialty, choose a doctor, and confirm a time slot. To optimize UX, a "Quick Booking" shortcut is integrated, allowing patients to bypass the specialty selection by directly choosing a Featured Doctor.

3. Checkout & Payment Flow: A dedicated post-examination route where patients can access pending invoices, process transactions via an external payment gateway, and generate a blockchain-anchored e-receipt.

2.Technology Stack

- Frontend : Built with Flutter, enabling the deployment of a high-performance, cross-platform application (Mobile and Web) from a single codebase, leveraging native-like UI components.
- Backend: Developed using Node.js and Express.js to establish a highly scalable, asynchronous RESTful API capable of efficiently handling concurrent clinical operations.
- Database: Exclusively utilizes MongoDB (NoSQL) alongside Mongoose ORM. Its flexible, document-oriented structure is specifically chosen to embed complex, dynamic medical records and unstructured AI metadata without the constraints of rigid schemas.
- Artificial Intelligence: Powered by a Python (Flask) backend integrating TensorFlow/Keras for deep learning (CNN-based dermatological analysis) and Scikit-Learn for machine learning (KNN-based diabetes prediction).
- Blockchain & Smart Contracts: Integrates the Ethereum ecosystem (deployed on the Sepolia testnet) to guarantee data integrity and transparent access control. Smart contracts are developed using Solidity and Hardhat, leveraging OpenZeppelin libraries for secure Role-Based Access Control (RBAC). To optimize storage and gas fees, the system employs IPFS (via Pinata/Infura) for off-chain storage of encrypted medical data, while only anchoring the cryptographic hashes (CIDs) on-chain. Furthermore, ethers.js is utilized across the frontend and backend as the primary SDK to interact with the blockchain, sign transactions, and actively listen to event audit logs.
- DevOps & Deployment: Employs Docker for service containerization and Kubernetes for orchestration, supported by automated CI/CD pipelines and centralized monitoring tools (ELK Stack, Prometheus) to guarantee system reliability.

Building the Minimum Viable Product (MVP)

To validate the system's feasibility and operational efficiency, the project follows the Minimum Viable Product (MVP) approach. This strategy prioritizes the development of critical workflows while temporarily excluding non-essential features (live chat, loyalty points) to ensure system stability and performance.

1. Core Features

- Patient Mobile Application: Includes user authentication, the Dual Booking Flow (Standard and Quick Booking via Featured Doctors), the AI Health Prediction module (for uploading dermatological images or inputting clinical metrics), and the secure Checkout/Payment flow.
- Doctor/Admin Web Dashboard: Facilitates appointment management, allows doctors to review AI-assisted diagnostic inferences (CNN/KNN results), and includes the critical function of finalizing medical records for blockchain anchoring.

2. Prototype & Deployment

The deployed prototype successfully integrates all distributed components of the

system:

- Frontend: A fully functional, cross-platform Flutter application (packaged as an Android APK for demonstration).
- Backend & AI Integration: The Node.js core server and the Python (Flask) AI microservices are seamlessly connected via RESTful APIs, successfully handling real-time data exchange and model inference.
- Blockchain Network: A Hyperledger Fabric network is successfully configured (locally/testnet) to generate and log immutable cryptographic hashes (Tx_Hash) immediately upon payment completion or medical record finalization.

3. Outstanding Value & USP

This MVP establishes a strong competitive advantage over traditional clinic management software through two primary Unique Selling Propositions (USPs):

- Intelligent Diagnostic Support: By incorporating Machine Learning and Deep Learning (KNN/CNN), the system provides preliminary health assessments. This empowers patients with instant insights and provides medical staff with a reliable, data-driven reference prior to the actual examination, significantly reducing clinical overload.
- Absolute Data Integrity (Digital Trust): Anchoring sensitive Electronic Medical Records (EMR) and financial e-receipts onto a distributed blockchain network guarantees tamper-proof security. This eliminates the risk of unauthorized data modification and establishes absolute digital trust between patients and the healthcare facility.